

SCALA PROGRAMMING: OPERATORS AND BOOLEAN VALUES



by
Dr M Rajasekhara Babu

School of
Computer Science and Engineering
<https://www.rajababu.org>

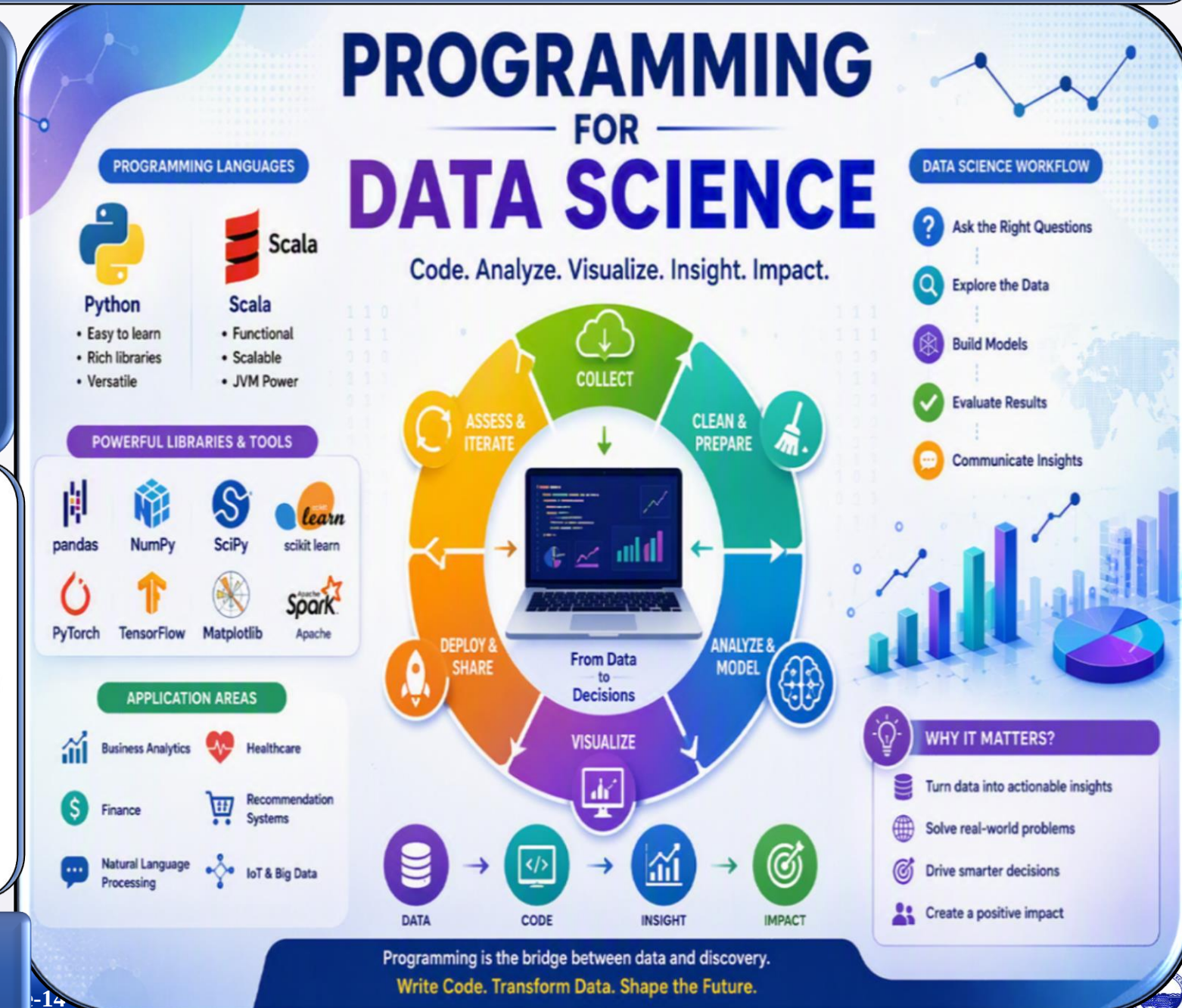


VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India





Learning Objectives

01

Understand Operators

Identify and use different types of operators in Scala.

02

Arithmetic & Relational

Perform calculations and compare values using operators.

03

Logical & Assignment

Combine Boolean conditions and assign/update values.

04

Build Programs

Evaluate Boolean expressions and develop decision-making programs.



What Is an Operator?

Definition

An operator is a symbol that performs an operation on one or more operands.
Types in Scala

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators

Quick Example

```
val a = 10  
val b = 5  
val sum = a + b
```

Here, **+** is the operator and a, b are the operands. The result stored in **sum** is 15.

Arithmetic Operators

Arithmetic operators are used for mathematical calculations on numeric values.

Operator	Description	Example (a=20, b=6)	Result
+	Addition	$a + b$	26
-	Subtraction	$a - b$	14
*	Multiplication	$a * b$	120
/	Division	a / b	3
%	Modulus (Remainder)	$a \% b$	2

Integer vs. Floating-Point Division

Integer Division

```
println(10 / 3)
```

Output: 3

When both operands are integers, Scala performs integer division — the decimal portion is truncated.

Floating-Point Division

```
println(10.0 / 3)
```

Output: 3.3333333333333335

When at least one operand is a **Double**, Scala returns a precise decimal result.

📌 Always use a decimal point (e.g., 10.0) when you need a fractional result from division.

Relational Operators

Relational operators compare two values and always return a Boolean result (true or false).

Operator	Meaning	Example (a=10, b=20)	Result
==	Equal to	a == b	false
!=	Not Equal to	a != b	true
>	Greater than	a > b	false
<	Less than	a < b	true
>=	Greater than or Equal to	a >= b	false
<=	Less than or Equal to	a <= b	true

Practical Example

```
val marks = 75  
println(marks >= 50) // Output: true – student has passed
```

What Is a Boolean?

A Boolean data type stores only two possible values: true or false.

Declaration

```
val result: Boolean = true  
val status: Boolean = false
```

Example: Eligibility Check

```
val age = 18  
val eligible = age >= 18  
println(eligible)
```

Output: true

The expression `age >= 18` evaluates to a Boolean, which is stored in `eligible` and printed directly.



Logical operators combine multiple Boolean expressions to form complex conditions.

AND &&

Returns true only if both conditions are true.

OR ||

Returns true if at least one condition is true.

NOT !

Reverses a Boolean value — true becomes false and vice versa.

Truth Table

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

Example: Voting Eligibility

```
val age = 20
val citizen = true
println(age >= 18 && citizen)
```

Output: true

Both conditions must be satisfied:
the person must be 18 or older and a
citizen.

Logical OR (||)

Returns **true** if at least one condition is true.

```
val attendance = 60
println(attendance >= 75 ||
attendance >= 60)
```

Output: **true**

The second condition (attendance >= 60) is true, so the whole expression is true.

Logical NOT (!)

Reverses a Boolean value.

```
val result = true
println(!result)
```

Output: **false**

The ! operator flips true to false and false to true.

Assignment Operators

Assignment operators assign or update variable values. Compound operators combine arithmetic with assignment for concise code.

Basic Assignment

```
var x = 10
```

Addition Assignment

```
x += 5  
// x is now 15
```

Subtraction Assignment

```
x -= 3  
// x is now 12
```

Multiplication Assignment

```
x *= 2  
// x is now 24
```

Division Assignment

```
x /= 2  
// x is now 12
```

Operator Precedence

Without Parentheses

```
val result = 10 + 5 * 2  
println(result)
```

Output: 20

Scala evaluates $5 * 2 = 10$ first (multiplication has higher precedence), then $10 + 10 = 20$.

With Parentheses

```
val result = (10 + 5) * 2  
println(result)
```

Output: 30

Parentheses override default precedence — $(10 + 5) = 15$ is evaluated first, then $15 * 2 = 30$.

📌 Use parentheses to make your intent explicit and avoid precedence-related bugs.

Program 1: Arithmetic Calculator

```
import scala.io.StdIn

object Calculator {
  def main(args: Array[String]): Unit = {
    print("Enter First Number: ")
    val a = StdIn.readInt()
    print("Enter Second Number: ")
    val b = StdIn.readInt()
    println("Addition      = " + (a + b))
    println("Subtraction   = " + (a - b))
    println("Multiplication = " + (a * b))
    println("Division      = " + (a / b))
  }
}
```

This interactive program reads two integers from the user and displays the results of all four basic arithmetic operations.



Voting Eligibility

```
object VotingEligibility {  
  def main(args:  
Array[String]): Unit = {  
    print("Enter Age: ")  
    val age = StdIn.readInt()  
    val eligible = age >= 18  
    println("Eligible to Vote:  
" + eligible)  
  }  
}
```

Uses a relational operator to determine voting eligibility.

Scholarship Eligibility

```
object Scholarship {  
  def main(args: Array[String]): Unit =  
  {  
    print("Enter Marks: ")  
    val marks = StdIn.readInt()  
    print("Enter Attendance: ")  
    val attendance = StdIn.readInt()  
    val eligible =  
      marks >= 80 && attendance >= 75  
    println("Scholarship Eligible: " +  
eligible)  
  }  
}
```

Combines two conditions using logical AND.

1

Activity 1: Precedence

Evaluate and record the outputs of:

$15 + 5 * 2$

$(15 + 5) * 2$

2

Activity 2: Relational Operators

Evaluate each

$20 > 10$

$20 == 10$

$20 != 10$

3

Activity 3: Voting Eligibility

Determine the result of $\text{age} \geq 18$ for ages:

- 16 → false
- 18 → true
- 21 → true



Arithmetic & Relational

Performed calculations and compared values using `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `>`, `<`, etc.



Boolean & Logical

Understood `true/false` values and combined conditions using `&&`, `||`, and `!`.



Assignment & Precedence

Applied compound assignment operators and used parentheses to control operator precedence.



1 Simple Calculator

Read two integers and display:
Addition, Subtraction,
Multiplication, Division, and
Modulus.

2 Pass/Fail Program

Read student marks. Display
true if marks ≥ 50 (Pass),
otherwise false (Fail) using a
Boolean result.

Assignment — Part 2

1 Eligibility Checker

Read Age and Citizenship Status
(true/false). Determine voting
eligibility using logical operators
(`&&`).

2 Expression Evaluation

Find the output of each expression
without running code first:

$10 + 20 * 3$

$(10 + 20) * 3$

$100 / 5 + 10$



1. Smart Scholarship Eligibility System

Problem Statement

A university awards scholarships based on the following criteria:

- Marks ≥ 85
- Attendance $\geq 80\%$
- Family Income $\leq ₹3,00,000$

Read all inputs from the user and determine:

1. Whether the student is eligible.
2. Number of conditions satisfied.
3. Scholarship category:
 - 3 conditions $\rightarrow$ Full Scholarship
 - 2 conditions $\rightarrow$ Partial Scholarship
 - Less than 2 $\rightarrow$ No Scholarship

Concepts Used

- User Input
- Operators
- Boolean Logic (&, ||)
- Variables

